



UnityCloud Troubleshooting Guide

Release 5.1.0

Date: April 2024

Affirmed Networks, Inc.

A Microsoft Company

Copyright© 2023 Affirmed Networks, Inc. All Rights Reserved.

AFFIRMED & DESIGN®, ACUITAS SERVICE MANAGEMENT SYSTEM®, AFFIRMED NETWORKS MOBILE CONTENT CLOUD™, AFFIRMED MOBILE CONTENT CLOUD™, UnityCloud™, UnityCloud PaaS™, UnityCloud Platform™, UnityCloud Operations™, UnityCloud Ops™, Operations and Policy Manager (OPM)™ and other trademarks and designs are the registered or unregistered trademarks of Affirmed Networks, Inc. and its subsidiaries in the United States and in foreign countries. All other trademarks are the property of their respective owners. The Affirmed Networks, Inc. trademarks may not be used in connection with any product or service that is not Affirmed Networks' in any manner that is likely to cause confusion among customers or in any manner that disparages or discredits Affirmed Networks, Inc.

This document contains information that is the property of Affirmed Networks, Inc. This document may not be copied, reproduced, reduced to any electronic medium or machine readable form, or otherwise duplicated, and the information herein may not be used, disseminated or otherwise disclosed, except with the prior written consent of Affirmed Networks, Inc.

Product(s) and features documented in this manual handle various forms of data relating to your users. You must comply with all laws and regulations applicable to your deployment, management, and use of said product(s), and you should take all appropriate technical and organizational measures to ensure you are handling this data properly according to any local legal and regulatory obligations.

You are responsible for determining whether said product(s) or feature(s) is/are appropriate for storage and processing of information subject to any specific law or regulation and for using said product(s) or feature(s) in a manner consistent with your own legal and regulatory obligations. You are also responsible for responding to any request from a third party regarding your use of said product(s), such as a request to take down content under the U.S. Digital Millennium Copyright Act or other applicable laws.

Contents

Preface	ix
Audience	ix
Release Information	ix
Conventions	ix
How to Obtain Product Documentation	x
Customer Support	x
Overview	xi
Prerequisites	xi
What this content does not cover	xi
Troubleshooting PaaS	1-1
Troubleshooting Consul	1-1
Functional Overview	1-1
Troubleshooting EFK	1-13
Troubleshooting ETCD	1-18
Deploymentor Runtime Failures	1-18
Data Collection	1-19
Data Analysis	1-20
System Recovery Process/Workaround	1-21
References	1-21
Troubleshooting Istio	1-21
Overview	1-22
Troubleshooting Kibana	1-31
Prerequisites	1-32
Issue Description	1-32
Possible Causes	1-32
Troubleshooting Steps	1-32
Workaround	1-32
Verification Steps	1-32
Prerequisites	1-32
Issue Description	1-32
Possible Causes	1-33
Troubleshooting Steps	1-33

Workaround	1-33
Verification Steps	1-33
Unable to Create anIndex Pattern from Kibana	1-33
Prerequisites	1-33
Issue Description	1-33
Possible Causes	1-33
Troubleshooting Steps	1-33
Workaround	1-33
Verification Steps	1-33
Possible Causes	1-34
Symptoms	1-34
Artifacts	1-34
Troubleshooting Steps	1-35
Troubleshooting Common Microservices	2-1
CHF Agent	2-1
Configuration Manager	2-1
GTPC Agent	2-1
CHF Agent	2-2
UsageReportNotSenttoor ReceivedbyCHFServer	2-2
Configuration Manager	2-4
Application Pods are inn-1/nState	2-4
Certificate Issuesfor RESTCONFRequests	2-6
Cfgmgr Failure	2-7
CLI LoginIssues	2-7
Config Update/Notification Not Received at an Application	2-9
IncorrectDisplay of CLI Commandor Help	2-10
GTPC Agent	2-11
Peer Initiated Traffic Dropped by Gtpcagent	2-11

List of Figures

Figure 1-1 Consul Cluster1-1

Figure 1-2 K8s Worker Node 1-2

List of Tables

Preface

This content is intended for use by support teams and customers who need to troubleshoot and maintain the UnityCloud system.

Audience

This guide is intended for all users who work with Affirmed Networks 5G or CNF products.

Release Information

See the Release Notes for important information, requirements, limitations, and restrictions that apply to this release.

Conventions

This guide uses the following conventions, when applicable:

Description	Convention	Examples
Command names, keywords, dialog boxes, buttons, icons, menu	Times New Roman bold font	Use configure_ha to configure... Enter y . To continue, press Enter .
Selecting a menu item	Menu => Option	Select Security => Group Management
Variable parameters for which you supply values	<courier bold italic>	configure_dr --host <host name>
User input	Courier bold font	cd /opt/Affirmed/NMS/bin
Screen messages, system output, sample source code	Courier regular font	result success
Required keywords and arguments in square brackets	[]	cluster [id] node [id] reboot
Optional keywords and arguments in curly brackets	{ }	{cdr-file-name}
Keywords separated by the pipe symbol. Requires you to specify only one item from the set.		Is this correct? [y n]
Keywords representing a single element in angle brackets	< >	<host name>

Description	Convention	Examples
Asterisk after a CLI object indicates a required object	*	name*
Variables, book and chapter titles, file path names, new terms, and emphasized text	<i>Times New Roman italic font</i>	See the <i>UnityCloud Troubleshooting Guide</i> . <i>Workflow</i> is an application running on the network element.
Images in online help		Many images can be enlarged by clicking the image with the left-mouse button. Alternately, they can be enlarged/reduced by pressing the Enter key when the image is highlighted (surrounded by a thick border) on the screen.

Note: Additional information that may apply to the subject text.

Tip: Helpful suggestions.

Caution: Proceed carefully to avoid possible equipment damage or data loss.

How to Obtain Product Documentation

Documentation is available for currently supported product releases. You can access Affirmed Networks Guides and Release Notes on the [Manuals](#) site. From here, you can also download a PDF version of an individual guide, or the full manual set for a product. For access and instructions, please contact Support.

Customer Support

Affirmed Networks provides technical support 24 hours a day, 7 days a week (24 x 7) for any customer with an active maintenance contract.

[Email support](#)

[Open a support ticket](#)

Call support

Local: +1-978-268-0800

Toll-free: +1-888-283-2838

Overview

This content is intended for use by support teams and customers who need to troubleshoot and maintain the UnityCloud system.

This guide includes troubleshooting topics of:

- [Troubleshooting PaaS](#)
- [Troubleshooting Common Microservices](#)

Prerequisites

- This content assumes that the reader has a good understanding of 3GPP networks and is familiar with Command Line Interface (CLI).
- Familiarity with UnityCloud and 5G Networks.

What this content does not cover

- Solution based integration content
- Holistic deployment instruction

***Note:** This content is a work in progress and will be built on release after release until troubleshooting content for all the Network Functions are covered.*

Troubleshooting PaaS

This section provides the troubleshooting topics for PaaS.

- [Troubleshooting Consul](#)
- [Troubleshooting EFK](#)
- [Troubleshooting ETCD](#)
- [Troubleshooting Istio](#)
- [Troubleshooting Kibana](#)

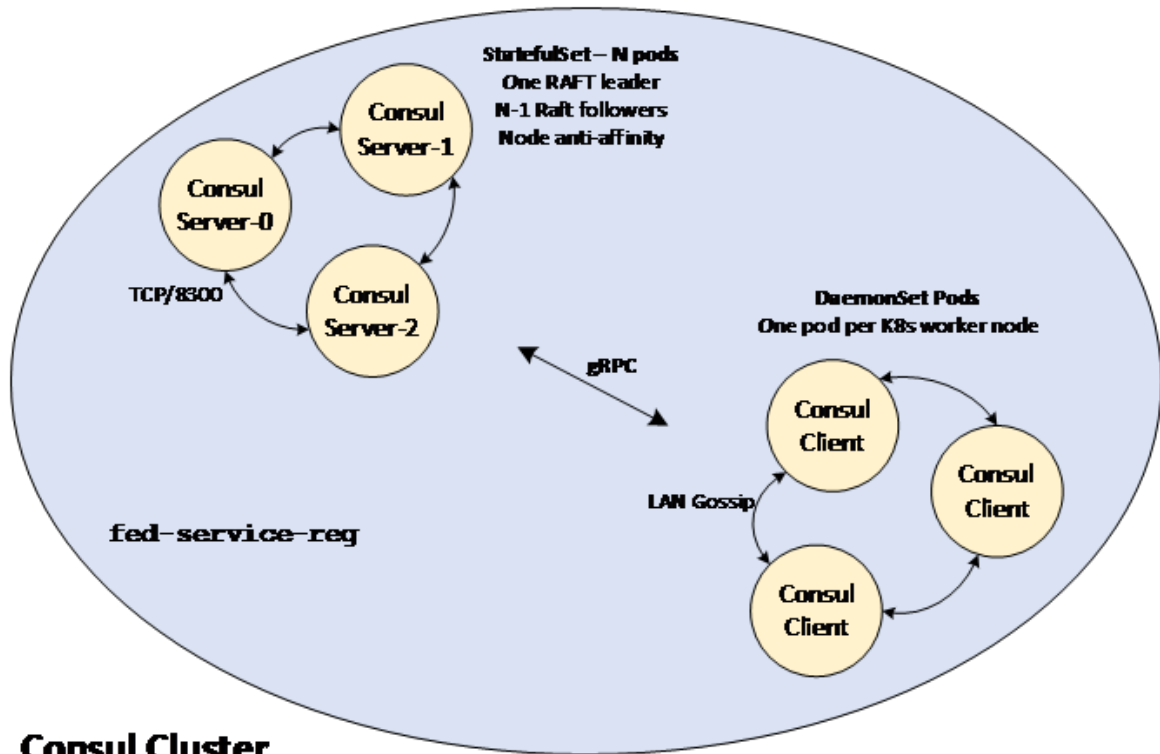
Troubleshooting Consul

Functional Overview

The Consul Service Registry is deployed on a K8s cluster in namespace fed-service-reg. The Consul Service Registry is comprised of a StatefulSet and a DaemonSet.

The StatefulSet consists of three replica pods named “Server-0, Server-1 and Server-2”. The pod anti-affinity rules for the StatefulSet pods ensures that each pod is scheduled on a discrete worker node.

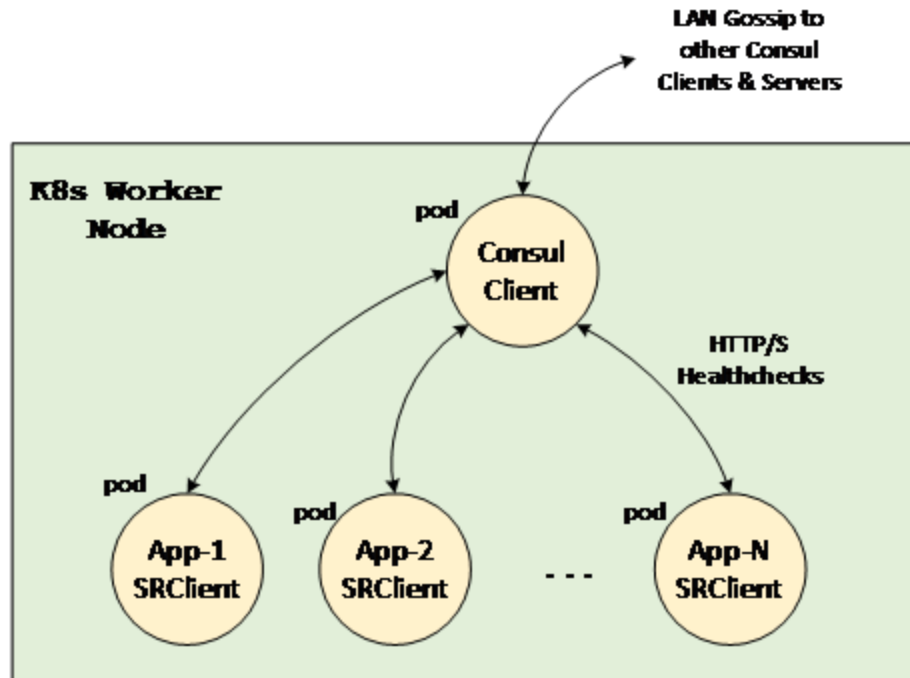
Figure 1-1 Consul Cluster



Consul Cluster

The DaemonSet places one Consul Client pod on each worker node in the K8s cluster. The SRClient in each application communicates with its node-local Consul client pod.

Figure 1-2 K8s Worker Node



One Consul Client DaemonSet Pod on each K8s Worker Node

Data Collection

SRClient Logs

To manually view the infra container logs for a pod, execute below commands:

```
# kubectl -n fed-smf get pods
# kubectl -n fed-smf logs POD_NAME -c infra
```

Consul Logs

To manually view the Consul Service Registry container logs, execute below commands:

```
# kubectl -n fed-service-reg get pods
# kubectl -n fed-service-reg logs POD_NAME -c consul
```

Kargo Collection

A Kargo log capture will be needed if fed-service-reg fails.

Execute the capture beginning 60m prior to the onset of the failure event. Capture logs from fed-service-reg and from the effected federation(s). The two examples provided below can be modified or extended to cover the affected federation(s).

To capture Kargo logs from fed-smf and fed-service-reg, execute below command from the masterVM.

```
# /usr/local/bin/kubect1-afn-kargo-collect -p fed-smf:kargo-  
default-profile,fed-service-reg:kargo-default-profile -d  
  
<duration-in-minutes> -s <UTC starttime in format- 2022-04-  
14T15:16:00.00Z>
```

To capture Kargo logs from fed-upf and fed-service-reg, execute below command from the masterVM.

```
# /usr/local/bin/kubect1-afn-kargo-collect -p fed-upf:kargo-  
default-profile,fed-service-reg:kargo-default-profile -d  
  
<duration-in-minutes> -s <UTC starttime in format- 2022-04-  
14T15:16:00.00Z>
```

Failure Modes

The following failure modes are associated with fed-service-reg.

Application Pod Liveness Failures

The pods within the top-level NF application federations improperly initialize if fed-service-reg fails to initialize properly. The primary application pod symptom is that all NF pods on a worker node are **NotReady** and are failing their K8s Liveness probes. These pods will generally have **low** uptime due to the frequent restarts.

This situation can affect a single node, multiple nodes, or all nodes. It is directly caused by the Consul client pod on that worker node not initializing properly. Typically due to an inability to correctly install the HostPort 8500 by the CNI plugin. See **HostPort** section for more information.

This failure scenario was only observed in releases that predate UC 2.3.0 where certain characteristics of the VmImage prevented proper initialization of the HostPort. This issue is addressed in all supported UC Production Releases.

Loss of Quorum

Loss of quorum amongst the Server-N pods may occur when there are multiple simultaneous worker node failures AND more than one Server-N pods reside on the failed worker nodes. The Service Registry requires that multiple Server- pods are operationally available to maintain quorum. If multiple Server-N pods are not simultaneously lost, quorum will remain stable.

The Service Registry will tolerate the failure of any single K8s worker node.

When Quorum is lost the servers will in some cases recover themselves. In other scenarios, a kind of split brain situation could develop where two different leaders exist.

Worker Node Resiliency

The loss of any worker node will cause the loss of the Consul client pod on that worker node. The client pods will return and rejoin the Consul Service Registry when the worker node returns.

No Operator intervention is required.

Consul Pod Failure

The Consul Service Registry will automatically recover from the loss of any client or Server-N pod.

Operator intervention is required.

Note: Note: When a client pod fails, all “ServiceName” published by applications on that node will be indicated as “Unavailable” until the client pod returns. At that time the “ServiceName” will again be indicated as “Available”.

Note: Note: When a server-N pod fails, there is no perturbation to the published “ServiceName”. The replacement pod simply rejoins the “server-N cluster”.

Consul Container Failure

If there are pods in fed-service-reg that indicate container restarts on the **kc get pods** output, then the **kubectl prior log** and a Kargo dump (see Kargo instructions) should be initiated.

To capture the containers prior log. The output is redirected to a /tmp file.

```
# kubectl -n fed-service-reg logs -p POD_NAME -c consul >
/tmp/consulPrevLogs.txt
```

Health Assessments

Assess the overall health of the pods in the fed-service-reg namespace.

1. Verify that fed-service-reg pods are deployed and in “Running” STATUS in accordance with the dimensioning parameters of the cluster (e.g. number of worker nodes). There shall be three “server-N” pods and one client pod per K8s worker node.

```
# kc -n fed-service-reg get pods -o wide
```

NAME	READY	STATUS			
RESTARTS	AGE	IP	NODE	NOMINATED	
NODE	READINESS	GATES			

```

fed-service-reg-pod-service-reg-5k6mr      1/1      Running
0      4d23h      192.168.147.108      node-1-a00044-
4b2c09f7b029c1aa      <none>      <none>

fed-service-reg-pod-service-reg-bt9vm      1/1      Running
0      4d23h      192.168.1.20      node-2-a00044-
4b2c09f7b029c1aa      <none>      <none>

fed-service-reg-pod-service-reg-mlbk9      1/1      Running
0      4d23h      192.168.60.56      node-3-a00044-
4b2c09f7b029c1aa      <none>      <none>

fed-service-reg-pod-service-reg-server-0      1/1      Running
0      4d23h      192.168.1.17      node-2-a00044-
4b2c09f7b029c1aa      <none>      <none>

fed-service-reg-pod-service-reg-server-1      1/1      Running
0      4d23h      192.168.147.103      node-1-a00044-
4b2c09f7b029c1aa      <none>      <none>

fed-service-reg-pod-service-reg-server-2      1/1      Running
0      4d23h      192.168.60.13      node-3-a00044-
4b2c09f7b029c1aa      <none>      <none>

```

Note: If some PODs are missing or not in “Running” STATUS, then K8s likely failed to deploy due to resource limitations, such as lack of CPU or RAM. This must be resolved before continuing by referring to the K8s Troubleshooting document.

Note: If the pods show large RESTART counts, then perform a Kargo log capture.

2. Validate Helm Variables.

There must be exactly three server pods (above) with names that end in “server-0”, “server-1” and “server-2”. The size of the StatefulSet is controlled by the Helm variables. The default value of “3” for these two variables is correct and should not be modified during the deployment of fed-service-reg.

```
# helm get values fed-service-reg -n fed-service-reg
```

USER-SUPPLIED VALUES:

global:

grafana:

url: http://10.15.25.95:3000

ingressNetworkPolicy:

enabled: true

ingressTrafficNamespaces:

- fed-example
- fed-paas-helpers
- fed-db-etcd
- fed-db-mongo-ram
- fed-istio
- fed-metallb
- fed-prometheus
- fed-amf
- fed-smf
- fed-mme
- fed-nrf
- fed-nssf
- fed-upf-sim
- fed-smf-sim
- fed-upf-sim2
- fed-upf-sim3
- fed-smf-test
- fed-upf-test
- fed-n3iwf
- fed-amf-test
- fed-agf

pod-service_reg:

server:

bootstrapExpect: 3

replicas: 3

3. Worker Node HostPort Configuration.

Verify the worker node HostPort configuration for each worker node. HostPort 8500 is used to connect to that node's Consul Client DaemonSet pod. This HostPort is installed in the iptables when the DaemonSet pod is deployed.

For each worker node IP address, assess that the Consul client returns a 200 OK. Additionally, verify that all Consul client pods return the same leader IP address. This curl command must succeed for each client pod (i.e. each Node IP Address) in the cluster

```
# curl -v http://10.15.10.X:8500/v1/status/leader; echo
```

4. Verify Consul Leader Health

For all three server pods, and for each client pod on a worker node, verify that they all return the same IP address for the leader.

```
kc -n fed-service-reg exec -it fed-service-reg-pod-service-  
reg-server-0 -c consul -- curl -v  
http://127.0.0.1:8500/v1/status/leader; echo
```

```
kc -n fed-service-reg exec -it fed-service-reg-pod-service-  
reg-5k6mr -c consul -- curl -v  
http://127.0.0.1:8500/v1/status/leader; echo
```

If there are inconsistent leaders reported by any of the three server pods, then quorum is lost. See below to restore quorum.

5. Verify Consul Peer Set

For all three servers, and for each client, verify that they all return the same set of IP addresses for the servers. There must be three server IP addresses returned. These correspond to the three server pod's POD_IPADDRESS.

```
kc -n fed-service-reg exec -it fed-service-reg-pod-service-  
reg-server-0 -c consul -- curl -v  
http://127.0.0.1:8500/v1/status/peers; echo
```

```
kc -n fed-service-reg exec -it fed-service-reg-pod-service-  
reg-5k6mr -c consul -- curl -v  
http://127.0.0.1:8500/v1/status/leader; echo
```

6. Verify that none of the Consul pods are using excessive CPU cycles.

Execute Top Pods of the fed-service-reg namespace. The output below is typical for a three node K8s cluster after all of the feds are deployed.

```
# kc -n fed-service-reg top pods  
  
NAME          CPU (cores)
```

MEMORY (bytes)

fed-service-reg-pod-service-reg-5k6mr	22m	32Mi
fed-service-reg-pod-service-reg-bt9vm	22m	26Mi
fed-service-reg-pod-service-reg-mlbk9	17m	28Mi
fed-service-reg-pod-service-reg-server-0	16m	43Mi
fed-service-reg-pod-service-reg-server-1	39m	51Mi
fed-service-reg-pod-service-reg-server-2	40m	42Mi

7. Verify SRClient Health.

The SRClient sidecar resides in all pod’s infra container. In all namespaces, SRClient should be free from any errors. The presence of “socket.timeout” errors may be indicative of a performance or networking issue. Operational issues are generally indicated by the presence of “ERROR:” logs emitted by SRClient.

This bash command will parse all SRClient logs, in all namespaces, for the pattern “ERROR:”.

```
# date; for nsitem in $(kubectl get namespaces --output
jsonpath={.items [*].metadata.name}); do echo $nsitem; if [[
$nsitem =~ 'kube' ]]; then continue; fi;

for podName in $(kubectl -n $nsitem get pods --output
jsonpath={.items [*].metadata.name}); do for cName in
$(kubectl -n $nsitem get pod $podName --output jsonpath=
{.spec.containers[*].name}); do if [[ $cName != 'infra' ]];
then continue; fi; echo "    $podName"; kubectl -n $nsitem
logs $podName -c $cName |grep 'ERROR:'; done; done;
done;    date
```

The above command may identify a handful of UNsubscribeService errors from mong-cfg-op. These can be ignored. Following is an example:

```
{"severity":"error", "debug-string":"[2022-06-09T19:26:49]
[SRClient] ERROR: UNsubscribeService, Failed. Persistent
Intercept. Service:nf-monitor-ns-fed-smf Client:mongo-cfg-
op_mongo_ram"}
```

An example of a “socket.timeout” error has this signature:

```
{"severity":"error", "debug-string":"[2022-05-30T10:26:17]
[SRClient] ERROR: getHealthService, except:<class
'socket.timeout'> reason:timed out
URL:http://10.163.67.37:8500/v1/health/service/configdata-
nf-inv-mgr-ns-fed-nrf?index=611&wait=10s"}
```

8. Verify LAN joined

Note: Note: *Approximately 30s to 40s after each Consul container starts, that container emits a LAN joined log message. This can be observed by examining the container logs. During the initial 30s startup, the application will identify peers and participate in elections (server pods only).*

Example log:

```
2022-06-09T19:22:34.157Z [INFO] agent: (LAN) joined: number_
of_nodes=3
```

Mitigations - Manual Intervention

HostPort 8500

When HostPort 8500 on a worker node is unreachable, the first mitigation is to verify that the client pod on that node is deployed and healthy.

If the client pod is healthy, then inspecting the iptables to ensure there is exactly one rule for HostPort is required.

These two commands can be used to observe the HostPort entries in the iptables. The Operator must execute these commands on the effected worker node (i.e. NOT the masterVM) and be logged in as user root. Privileged root access is required to get the correct “nft” output.

```
# nft list table ip nat |grep -B2 -A2 8500

# nft list table ip6 nat |grep -B2 -A2 8500
```

For each command, there should be exactly one CNI-HOSTPORT-DNAT and exactly one chain where the port is included. Here is the example output.

```
# nft list table ip nat |grep -B2 -A2 8500

chain CNI-HOSTPORT-DNAT {

meta l4proto tcp      tcp dport { 8500,8502} counter packets
5879261 bytes 352755660 jump CNI-DN- 75d5fc2a7701bcc73a1be

}

chain CNI-DN-75d5fc2a7701bcc73a1be {

meta l4proto tcp ip saddr 192.168.147.108 tcp dport 8500
counter packets 0 bytes 0 jump CNI-HOSTPORT- SETMARK

meta l4proto tcp ip saddr 127.0.0.1 tcp dport 8500 counter
packets 0 bytes 0 jump CNI-HOSTPORT- SETMARK

meta l4proto tcp tcp dport 8500 counter packets 5879261
bytes 352755660 dnat to
192.168.147.108:8500
```

```
meta l4proto tcp ip saddr 192.168.147.108 tcp dport 8502
counter packets 0 bytes 0 jump CNI-HOSTPORT- SETMARK

meta l4proto tcp ip saddr 127.0.0.1 tcp dport 8502 counter
packets 0 bytes 0 jump CNI-HOSTPORT- SETMARK

# nft list table ip6 nat |grep -B2 -A2 8500 chain CNI-
HOSTPORT-DNAT {

meta l4proto tcp      tcp dport { 8500,8502} counter packets 0
bytes 0 jump CNI-DN-75d5fc2a7701bcc73a1be

}

chain CNI-DN-75d5fc2a7701bcc73a1be { meta l4proto tcp ip6
saddr

fd33:8ef:1802:6cbf:f297:7f62:9252:9370 tcp dport 8500
counter packets 0 bytes 0 jump CNI-HOSTPORT-SETMARK

meta l4proto tcp tcp dport 8500 counter packets 0 bytes 0
dnat to [fd33:8ef:1802:6cbf:f297:7f62:9252:9370]:8500

meta l4proto tcp ip6 saddr
fd33:8ef:1802:6cbf:f297:7f62:9252:9370 tcp dport 8502
counter packets 0 bytes 0 jump CNI-HOSTPORT-SETMARK


meta l4proto tcp tcp dport 8502 counter packets 0 bytes 0
dnat to [fd33:8ef:1802:6cbf:f297:7f62:9252:9370]:8502

OR a more concise command

# nft list table ip nat |grep -A2 'chain CNI-HOSTPORT-DNAT'

chain CNI-HOSTPORT-DNAT {

meta l4proto tcp      tcp dport { 8500,8502} counter packets
843017 bytes 50581020 jump CNI-DN- 0f6a36f4c64846cc2eb23

}
```

If the curl commands to a **NodeIP:8500** continue to fail, then delete the client pod on that worker node.

If the pod returns and remains unreachable at NodeIP:8500 AND the iptables output deviates from the examples above, then the mechanism required to address the issue is to reboot the K8s worker node.

Note: Note: While it is possible (as root) to edit the iptables directly, that procedure is error prone and not recommended.

Restoring Quorum Set of Three

Rebuilding the Consul Cluster with a quorum set of three.

Normally this should not be necessary, but if multiple worker nodes in the cluster have been simultaneously rebooted, quorum within the Consul Service Registry server set may have been lost.

The recommended recovery is:

- uninstall all application federations above fed-service-reg
 - Refer to *UnityCloud Deployment Guide* for all federations that were deployed after fed-service-reg
- uninstall fed-service-reg
- install fed-service-reg
- install the application federations

Useful Debugging Tools

Display a ServiceName publication

The commands mentioned below are utilized to display the “publications” for a specific ServiceName in a namespace. The namespace and the ServiceName are used to identify the ServiceID instances that were published. A specific ServiceID is then used to display the ServiceAttributes.

These commands must be executed on the masterVM. The IP Address specified is any one of the K8s Worker Node IP address. The binaries “curl”, “base64” and “jq” are present in the MasterVM image.

List all instances of the “timer” ServiceName in namespace “fed-smf”. Note the concatenated “ns” format of the ServiceName used internally by SRClient/Consul.

```
# curl -v http://10.15.10.248:8500/v1/catalog/service/timer-ns-fed-smf |jq .
```

List the available ServiceID instances for this ServiceName.

```
# curl -v http://10.15.10.248:8500/v1/catalog/service/timer-ns-fed-smf |jq '.[].ServiceID'
```

Display a single ServiceID instance of this ServiceName from the ServiceRegistry Catalog.

```
# curl --get http://10.15.10.248:8500/v1/catalog/service/timer-ns-fed-smf --data-urlencode 'filter=ServiceID=="timer-fed-smf-chronos-1-chronos-vprobeagent"' |jq .
```

Display the ServiceAttributes of the single ServiceID instance of this ServiceName.


```
# curl --get
http://10.15.10.248:8500/v1/catalog/service/timer-ns-fed-smf

--data-urlencode 'filter=ServiceID=="timer-fed-smf-chronos-
1-chronos-vprobeagent"' |jq -r '.[0].ServiceTags[0]' |base64

-d; echo; echo
```

Display a Persistent Publication

The Persistent Publications (e.g. nf-monitor, mongo-cfg-op_mongo_ram, etcd-monitor) are stored in the Consul Key-Value store. These publications are not stored in the Consul ServiceRegistry Catalog.

The commands below can be utilized to list and identify the persistent publications and their attributes. The HTTP query parameter “recurse” is used to display the “key” values deeper in a path that share the specified path prefix. The identified “key” is then used to display a specific entry.

These commands are executed on the MasterVM. The IP Address specified is any K8s Worker Node. These two commands will display the full set of persistent publications:

```
# curl -v http://10.15.10.247:8500/v1/kv/azure-mobile-
core?recurse |jq .

# curl -v http://10.15.10.247:8500/v1/kv/azure-mobile-
core/persistent-pubs?recurse |jq .
```

Display the available “key” path values for each Persistent Publication.

```
# curl -v http://10.15.10.247:8500/v1/kv/azure-mobile-
core/persistent-pubs?recurse |jq '.[].Key'
```

Display the contents of a specific “key”.

```
# curl -v http://10.15.10.247:8500/v1/kv/azure-mobile-
core/persistent-pubs/nf-monitor-ns-fed-smf/nf-monitor_fed-
smf |jq .
```

The “Value” field is b64encoded. Display the decoded “Value” field.

```
# curl -v http://10.15.10.247:8500/v1/kv/azure-mobile-
core/persistent-pubs/nf-monitor-ns-fed-smf/nf-monitor_fed-
smf |jq -r '.[].Value' |base64 -d; echo; echo
```

Troubleshooting EFK

This section provides troubleshooting topics for EFK.

Application Index is not available in ElasticSearch

This section describes the procedure to resolve an error that says application index is not available.

Prerequisites

None

Issue Description

None

Possible Causes

None

Troubleshooting Steps

1. Check the configmap in the respective application's namespace.

```
kc get configmap -n fed-example
```

Example:

```
$ kc get configmap -n fed-example
```

```
NAME DATA AGE
```

```
db-nosql-ram-mongo 1 13d
```

```
empty-map 1 13d
```

```
fluentd-config 1 13d
```

```
tabdb-etcd 1 13d
```

2. Check the external IP and port of the ElasticSearch by describing the configmap.

```
$ kubectl describe configmap fluentd-config -n fed-example
```

3. Check in the description of the configmap whether the Elasticsearch IP and port are correctly configured.

```
<store>
```

```
@type elasticsearch
```

```
host 10.15.10.161
```

```
port 9200
```

```
logstash_format true
```

Workaround

None

Verification Steps

None

ElasticSearch is in Failed/Pending/Error/Creating State

Prerequisites

None

Issue Description

ElasticSearch is in Failed/Pending/Error/Creating State.

Possible Causes

During ElasticSearch deployment, the resources may be incorrectly configured.

Troubleshooting Steps

1. Check the status of the pods.

```
kc get pods -n fed-elastic
```

Example:

```
$ kc get pods -n fed-elastic
```

NAME	READY	STATUS	RESTARTS	AGE
client-fed-elastic-0	2/2	Pending	0	64s

2. Check the status of the ElasticSearch CRO.

```
kubect1 get elasticsearch -n fed-elastic
```

Example:

```
$ kubect1 get elasticsearch -n fed-elastic
```

NAME	VERSION	STATUS	AGE
fed-elastic	6.5.3	Failed	57s

Workaround

When the status of Elasticsearch CRO is in the failed state, then do not purge the helm charts. Delete the Elasticsearch CRO alone as shown below:

```
kc delete elasticsearch fed-elastic -n fed-elastic
```

Example:

```
$ kc delete elasticsearch fed-elastic -n fed-elastic
```

```
elasticsearch.kubedb.com "fed-elastic" deleted
```

```
$ kubect1 get elasticsearch -n fed-elastic
```

```
No resources found.
```

Verification Steps

Delete the helm-charts and then redeploy the fed-elastic with the correct configuration.

ElasticSearch is not Deployed

This section describes the procedure to resolve failure of ElasticSearch deployment.

Prerequisites

None

Issue Description

None

Possible Causes

None

Troubleshooting Steps

Deploy the ElasticSearch and delete the fluentd pods in the fed-paas-helpers namespace to discover the ElasticSearch.

Workaround

None

Verification Steps

None

ElasticSearch Status to Critical

A failure of the ElasticsearchOpsRequest will set the ElasticSearch status to Critical. In such situations, perform the following operation to bring back the ElasticSearch to the stable state:

Prerequisites

None

Issue Description

A failure of the ElasticsearchOpsRequest will set the ElasticSearch status to Critical.

Possible Causes

A failure of the ElasticsearchOpsRequest will set the ElasticSearch status to Critical.

Troubleshooting Steps

1. Check the ElasticSearch status:

```
# kubectl get elasticsearch -n fed-elastic
```

NAME	VERSION	STATUS	AGE
fed-elastic	7.9.3-searchguard	Critical	10h

2. Edit the Elasticsearch CRO and modify the pods and resources in alignment with the current system configuration to match:
 - a. the exact number of pods in case of horizontal scaling
 - b. the exact resources in case of vertical scaling

Example:

```
# kc edit es fed-elastic -n fed-elastic
```

```
topology:
  data:
    replicas: 3 <== Modify to correct number of data pods
    resources:
      limits:
        cpu: "1" <== Modify to correct CPU
        memory: 1Gi <== Modify to correct Memory
      requests:
        cpu: 900m <== Modify to correct CPU
        memory: 1Gi <== Modify to correct Memory
    storage:
      accessModes:
        - ReadWriteOnce
      resources:
        requests:
          storage: 150Gi
        storageClassName: cinder-class-es
    suffix: data
```

Workaround

None

Verification Steps

Restart the enterprise and operator pods of fed-kubedb-operator and then wait for the pods to come up. This will bring the Elasticsearch status back to the Ready state.

```
# kubectl get pods -n fed-kubedb-operator
```

```
NAME          READY   STATUS
```

```
RESTARTS     AGE
```

```
kubedb-autosc-86dcb689f-gxspq    1/1    Running    0 48m
```

```
kubedb-enterprise-bf6855f4b-1f7qh 1/1    Running    4 48m
```

```
kubedb-operator-54c497f684-6qh2c 1/1    Running    0 48m
```

```
# kubectl delete pod kubedb-enterprise-bf6855f4b-1f7qh -n
fed-kubedb-operator
```

```
pod "kubedb-enterprise-bf6855f4b-1f7qh" deleted
```

```
# kubectl delete pod kubedb-operator-54c497f684-6qh2c -n
fed-kubedb-operator

pod "kubedb-operator-54c497f684-6qh2c" deleted # kubectl get
elasticsearch -n fed-elastic
NAME          VERSION          STATUS          AGE
fed-elastic    7.9.3-searchguard  Ready          10h
```

Fluentd is Unable to Send Logs to ElasticSearch

This section describes the procedure to resolve Fluentd is unable to send logs to ElasticSearch.

Prerequisites

None

Issue Description

Fluentd is unable to send logs to ElasticSearch due to incorrect configuration.

Possible Causes

None

Troubleshooting Steps

Enable the logging field in the required fed-<application>/interface/values.yaml.

Workaround

None

Verification Steps

Try sending logs to ElasticSearch.

Troubleshooting ETCD

Deployment or Runtime Failures

This section describes the possible issues with ETCD in UC clusters and provides steps to recover. The logs are collected offline for analyzing the root cause.

The ETCD issues are mainly categorized into two types:

- ETCD dependent component failures
- ETCD server failures

ETCD Dependent Component Failures

The following are the possible component failures identified:

- K8s failed to deploy Etcd operator

- Etcd-operator request to deploy ETCD pods failed
- ETCD pod deployment failure because of resource unavailability
- ETCD init container DNS look up failure
- Helm post install taking too long

ETCD Server Failures

The following are the possible ETCD server failures identified:

- ETCD pod terminated because of OOM
- ETCD pod pre-empted by other high priority pod
- ETCD loses quorum or frequent leader changes
- ETCD monitor publication issues

Issue Description

If any NF is inactive due to ETCD, verify with nf-monitor service within the federation. To get the status of ETCD and other critical service with fed-upf, execute the following command:

```
kc exec -it <upfmonitor-pod-name> -n fed-upf -c upfmonitor -
- curlhttp://127.0.0.1:9090/nf-monitor/v1/nf-info
(responsebody will be injson) | python -mjson.tool
```

Note: Note: If the ETCD is inactive, identify if the ETCD is down due to server issue or etcd dependent components' issue.

Symptoms of ETCD Failure

The following are the symptoms of ETCD failure:

- etcd-cluster-* pods not deployed or running in fed-db-etcd namespace
- No etcd-operator pod running in fed-db-etcd namespace
- Frequent restart of etcd-cluster pods
- Etcd-monitor pod in fed-paas-helpers is not deployed or running

Troubleshooting Steps

Data Collection

Follow the below steps to collect the data logs:

1. Collect the kargo profiles configured in fed-db-etcd:

```
"kubectlafnkargo_collect-pfed-db-etcd-d<duration>- s
<starttime>"
```
2. Collect the logs from all the pods running in the fed-db-etcd during the time of failure.

3. Collect **kc describe deploy etcd-operator -n fed-db-etcd**.
4. Collect **kc get events -n fed-db-etcd**.
5. Capture the screenshots for the memory and cpu dashboards of **fed-db-etcd** pods from the **kube-prom-grafana**.
6. If any of the etcd-cluster pods is running, collect the following output: **kc get etcdcluster -o yaml -n fed-db-etcd**.
7. Collect the logs from **etcd-monitor** pod in **fed-paas-helpers.h3**.

Data Analysis

Once the data is available, identify if the issue is due to the ETCD server failure or ETCD dependent components' failure to recover the system.

If the issue is due to dependent components (VM hardware, Kubernetes failure issues) then, fix them before recovering the cluster.

Look at the following data to identify if the issue is due to the dependent components:

- etcd-operator pod is not deployed – Kubernetes is having issues and hence check the logs and metrics of the kubernetes.

Execute the following command to display the possible reasons for the deployment failure:

```
"kc describe deploy etcd-operator -n fed-db-etcd"
```

- etcd-operator failed to deploy the etcd-cluster pods – check if the pod deployment request by the operator is rejected by kubernetes.
 - Verify logs of the etcd-operator and “kc get etcdcluster -o yaml -n fed-db-etcd” for the error messages.
 - If the failure is because of opa mutation webhook failure, then verify and fix opa pod in fed-opa before recovering ETCD.
 - If the failure is because of resource unavailability – we need to increase the memory and cpu for the nodes or clear other low priority microservices.
- If etcd-cluster pod is deployed but not in ready state or hanging in init-container – DNS look up in the init container has failed (verify init container logs to confirm) and fix the DNS issues.
- Confirm no networking issues – grep for following from the etcd-cluster pod logs:
 - “starting a new election at term” – too many of these
 - “failed to send out heartbeat on time” – too many of these
 - “snapshotting is taking more than seconds to finish” – too many of these.
 - dial tcp :2379: getsockopt: connection refused or dial tcp :2380: getsockopt: connection refused
- There are other error etcd logs related to slow disk but theoretically is not an issue.

— “apply entries took too long”

- If all the above checks are passed, then verify for any problem with etcd health status by executing the following command:

```
Kcexec-it<etcd-cluster-*>-cetcd-nfed-db-etcd-sh-c "etcdctl  
endpoint health"
```

- If the above response indicates failure then save the logs offline for root cause analysis and proceed for System Recovery Process/Workaround.
- If the database is healthy, but etcd-monitor (in fed-paas-helpers) has problems in identifying the issues and publishing to consul. There are two possibilities for the failure:
 - Etcd-monitor failing to connect with etcd database – check network connectivity and fix this.
 - Fed-service-reg is not ready and etcd-monitor failed to publish service availability. (Fix this, follow fed-service-reg troubleshoot guide).

System Recovery Process/Workaround

After the data is analyzed, follow below steps to recover etcd (if etcd dependent components issues exists fix them first and then perform below steps):

1. If some etcd pods are up, first delete all etcd pods using:
kubectl delete pod -n fed-db-etcd <pod-name>
2. Edit etcd-operator deployment and change spec.replicas to "0":
kubectleditdeployment-nfed-db-etcd fed-db-etcd-pod-etcd-operator-etcd-operator
3. Save the CRO "etcdcluster" yaml to /tmp and delete the CRO and redeploy it:
**kubectl get etcdcluster -n fed-db-etcd -o yaml >
/tmp/etcdcluster.yaml**
kubectl delete etcdcluster -n fed-db-etcd etcd-cluster
4. Edit /tmp/etcdcluster and remove the data in the status subsection.
kubectl create -f /tmp/etcdcluster.yaml
5. Edit the etcd-operator deployment replica back to "1":
kubectleditdeployment-nfed-db-etcd fed-db-etcd-pod-etcd-operator-etcd-operator

References

<https://etcd.io/docs/v3.4/op-guide/>

Troubleshooting Istio

This troubleshooting section applies to UC 3.1.1 and later, using Istio 1.11.2 and later.

Overview

The Istio Service Mesh provides low level pod to pod connectivity features. The Istio Service Mesh includes the IngressGW and EgressGW pods.

The Istio control plane pods are deployed in the fed-istio namespace. The UC fed-istio Helm chart instantiates the istio-operator pod and the IstioOperator CRO resources. The istio-operator pod reads each “istiooperator” resource instances and instantiates the prescribed Istio resources. This istio-operator pod is responsible for deploying the Service Mesh.

Subsequent NF Helm charts may also include IstioOperator CRO resources. When these are installed the istio-operator will create the resources described therein. This is the mechanism by which IngressGW and EgressGW pods are instantiated in NF namespaces.

The istiod pods are instantiated by the istio-operator. The istiod pods are responsible for propagating Service Mesh updates to each istio-proxy instance in the Service Mesh. The istiod pods are responsible for Service Mesh configuration.

The IngressGW pods are responsible for receiving request traffic from outside the cluster and load balancing that traffic to the appropriate Service endpoints within the cluster. The responses are sent back over the same TCP connections. The IngressGW pods are deployed within the namespace that they serve.

The EgressGW pods are responsible for sending request traffic to hosts outside of the cluster. The responses will transit the same TCP connections back into the cluster. Any security requirements (e.g. mTLS) of the external host is handled by the EgressGW. Each host external to the cluster is described by an ExternalHost resource.

Application pods have an istio-proxy container that is auto-injected into them. This container is the “istio-proxy”. Note that multiple application pods have disabled istio-proxy auto-injection. This disablement is UC release and application pod dependent.

Pre-requisites

Multus

Istio may be deployed in K8s Clusters with Multus or without Multus. If Multus will be deployed in the cluster by one of the NFs, then Istio MUST be deployed with its global.multusDeployed Helm value set to “true”.

If Multus is not to be deployed by any NF in the cluster, then Istio MUST NOT be deployed with its Multus flag set to true. The Multus “false” value is the default fed-istio Helm value. No Helm value for Istio Multus need to be set in the case where the cluster is not using Multus. This is the case when deploying a SMF cluster.

If Multus is deployed by an NF, then the global.multusDeployed Helm variable must be set to “true” for the Istio Helm chart. Any NF that deploys fed-sriovdp will have Multus deployed. This is the case when deploying a UPF cluster.

The use of Multus in the cluster by any NF requires the Istio Helm value `global.multusDeployed=true` to be set in `fed-istio's values.yaml` or `--set global.multusDeployed=true` can be passed in the `fed-istio's helm install` command.

Istio-Proxy Auto-Injection

Istio-Proxy auto-injection within a namespace is controlled by the “`istio-injection=enabled`” namespace label. The default behavior for a namespace is not to perform the istio-proxy auto-injection. Istio-proxy auto-injection can be controlled on a per pod basis (versus per namespace) using an annotation in the Deployment/StatefulSet resource specification.

Namespace Label Injection Control

A namespace can be labeled such that the default behavior in that namespace is to include istio-proxy sidecar auto-injection. The default behavior for a namespace is not to perform auto-injection. To show the namespaces and their associated labels use this command.

```
# kc get namespaces --show-labels
```

```
NAME      STATUS    AGE      LABELS
default    Active    5d22h
kubernetes.io/metadata.name=default

fed-crds    Active    5d21h    kubernetes.io/metadata.name=fed-
crds,name=fed-crds

fed-db-mongo-ram    Active    5d21h
kubernetes.io/metadata.name=fed-db-mongo-ram,name=fed-db-
mongo-ram,openpolicyagent=enabled,prometheus=enabled

fed-istio    Active    2d11h    istio-
injection=disabled,istio-operator-
managed=Reconcile,kubernetes.io/metadata.name=fed-
istio,name=fed-istio,prometheus=enabled

fed-smf    Active    5d20h    capture- agent=enabled,istio-
injection=enabled,kubernetes.io/metadata.name=fed-
smf,name=fed-smf,openpolicyagent=enabled,prometheus=enabled

fed-smf-test    Active    5d20h    capture-
agent=enabled,istio-
injection=enabled,kubernetes.io/metadata.name=fed-smf-
test,name=fed-smf-
test,openpolicyagent=enabled,prometheus=enabled
```

Pod Level Injection Control

A pod can contain an annotation that specifies the pod’s istio-injection preference. This mechanism is sometime used to disable auto-injection for specific pods in namespaces where the default is set to enabled through a namespace label.

This command is used to show the pods that are annotated.

```
# for nsitem in $(kubectl get namespaces --output jsonpath=
{.items[*].metadata.name}); do echo $nsitem; for podName in
$(kubectl -n $nsitem get pods --output jsonpath={.items
[*].metadata.name}); do echo $podName; kubectl -n $nsitem
get pod $podName --output jsonpath={.metadata.annotations} |
jq . |grep 'sidecar.istio.io/inject'; done; done
```

Failure Modes

The following failure modes are associated with Istio/fed-istio:

Istio-CNI pods not deployed

The “istio-cni-node-xyz” pods do not deploy. The pods are created by a DaemonSet. It is expected that each K8s worker node has one “istio-cni-node” pod. The DaemonSet is deployed by the “istio-operator” pod.

Improper IstioOperator Values

Helm values are used to populate or override values in the IstioOperator CRO. If an invalid value is specified, the istio-operator pod may not accept the value and fail to load the IstioOperator CRO.

OOM in Istiod / Istio-Operator pods

If the size of the cluster and the size of the Service Mesh combine to create an Istio configuration that does not fit in the memory footprint defined for these pods, one of these pods may take an OOM error and enter a pod/container restart cycle.

The current sizing has been tested on clusters of various large sizes. This error is unlikely unless Helm variables are overridden to intentionally constrain the Istio memory footprint.

OOM in Istio-Proxy Containers

The istio-proxy injected into multiple pods is configured with a generous memory limit. This limit should never be reached. If it is, the istio-proxy container may restart, with minimal impact to the pods function. Some pods do contain an annotation to modify (e.g. increase) the memory limit of the injected istio-proxy container.

External Host is unreachable

Multiple applications (e.g. li-x1, li-x2) rely on secure access to “cluster external” hosts. These applications create a mixture of static and dynamic ExternalHost CRO entries through istio-cfg-agent and istio-cfg-controller. Issues related to external host access are generally configuration or certificate based.

Health Assessment

To assess the overall health of the resource CROs and pods in the fed-istio namespace, perform the following steps:

1. Verify that the fed-istio pods are deployed and in **“Running”** Status in accordance with the dimensioning parameters of the cluster (For example: number of worker nodes). There can be an istio-cni pod on each K8s worker node, one istio-operator pod, and two istiod pods in the cluster.

```
# kc -n fed-istio get pods
```

NAME	READY	STATUS	RESTARTS	AGE
istio-cni-node-7kj6r	1/1	Running	0	13h
istio-cni-node-nwfjd	1/1	Running	0	13h
istio-cni-node-s8rx6	1/1	Running	0	13h
istio-cni-node-sr74b	1/1	Running	0	13h
istio-operator-6f59fbdd79-zvptv	1/1	Running	0	36h
istiod-67b88d5fb-tvzlt	1/1	Running	0	13h
istiod-67b88d5fb-vzphg	1/1	Running	0	13h
kiali-67c8f94577-vbtmn	1/1	Running	0	12h

If some PODs are missing or not in “Running” status, then K8s had failed to deploy due to resource limitations, such as lack of CPU or RAM. This must be resolved before continuing by referring to the K8s Troubleshooting document.

Note: If any of the pods display large RESTART counts, then perform a Kargo log capture.

2. Verify primary istiooperator resource.

Verify that the “istiooperator” resource has a status of **HEALTHY**. If the status is NOT **HEALTHY**, debug this issue. This may be because of an invalid Helm value format or invalid Helm value.

```
# kc -n fed-istio get istiooperator
```

NAME	REVISION
unitycloud-3-2-0-221-main-int-skumar-3-3-0-dbg	HEALTHY
	36h

3. Validate Helm Variables.
4. Verify that the multusDeployed value matches the cluster’s Multus configuration.

```
# helm get values -n fed-istio fed-istio
```

```
USER-SUPPLIED VALUES:
```

```
global:
```

```
grafana:
url: http://10.15.25.95:3000 ingressNetworkPolicy: enabled:
true ingressTrafficNamespaces:
- fed-service-reg
- fed-db-etcd
- fed-db-mongo-ram
- fed-metallb
- fed-prometheus
- fed-amf
- fed-smf
- fed-upf
- fed-nrf
- fed-nssf
- fed-n3iwf
- fed-grafana
- fed-kibana
- fed-paas-helpers
- fed-jaeger
- fed-mme
- fed-example
- fed-upf-sim
- fed-upf-sim2
- fed-upf-sim3
- fed-amf-test
- fed-smf-test
- fed-upf-test
- fed-smf-sim
- fed-kube-prom
- fed-locust
- fed-test-host
- fed-agf
```

```
- fed-elastalert
- fed-amf-mme
istioUpgradedVersion:
enable: false
multusDeployed: false
telemetry:
cpuRequest: 200m
pod-istio:
istiopilot:
```

5. Verify that istio-operator loaded all of the IstioOperator files and that no errors exist.

```
# kc -n fed-istio get pods
```

```
# kc -n fed-istio logs <Istio-operator Pod> -c istio-
operator
```

6. Verify that Gateway IstioOperator resources are healthy.

Get the istiooperator resources from all namespaces using the command **kc get istiooperator -A**.

The STATUS for all instances of istiooperator must be **HEALTHY**. If a resource instance is NOT HEALTHY then inspect the fed-istio istio-operator pod logs.

7. Describe the specific istiooperator resource description for any errors or events. Ensure to view the resource in its correct namespace.

```
# kc get istiooperator -A
```

```
NAMESPACE NAME
REVISION STATUS AGE
fed-example gateways-egress-fed-example-3.2.0-162-main- int-
dbg HEALTHY 3d5h
fed-example gateways-ingress-fed-example-3.2.0-162- main-
int-dbg HEALTHY 3d5h
fed-grafana gateways-ingress-fed-grafana-3.2.0-162- main-
int-dbg HEALTHY 2d1h
fed-istio unitycloud-3-2-0-221-main-int-skumar-3-3-0- dbg
HEALTHY 36h
fed-kube-prom gateways-ingress-fed-kube-prom-3.2.0-162-
main-int-dbg RECONCILING 3d5h
fed-prometheus gateways-ingress-fed-prometheus-3.2.0-3- rel-
3-2-0-dbg HEALTHY 4d22h
fed-smf-test gateways-egress-fed-smf-test-3.2.0-3-rel-3- 2-
0-dbg HEALTHY 4d22h
fed-smf-test gateways-ingress-fed-smf-test-3.2.0-3-rel- 3-2-
```

```

0-dbg HEALTHY 4d22h
fed-smf gateways-egress-fed-smf-3.2.0-3-rel-3-2-0- dbg
HEALTHY 4d22h
fed-smf gateways-egress-li-fed-smf-3.2.0-3-rel-3-2- 0-dbg
HEALTHY 4d22h
fed-smf gateways-ingress-fed-smf-3.2.0-3-rel-3-2-0- dbg
HEALTHY 4d22h
fed-smf gateways-ingress-li-fed-smf-3.2.0-3-rel-3- 2-0-dbg
HEALTHY 4d22h
fed-upf-sim gateways-egress-li-fed-upf-sim-3.2.0-3-rel- 3-2-
0-dbg HEALTHY 4d22h
fed-upf-sim gateways-ingress-li-fed-upf-sim-3.2.0-3- rel-3-
2-0-dbg HEALTHY 4d22h

# kc -n fed-kube-prom describe istiooperator gateways-
ingress-fed-kube-prom-3.2.0-162-main-int-dbg

# kc -n fed-istio get pods

# kc -n fed-istio logs istio-operator-6f59fbdd79-zvptv -c
istio-operator

```

8. Verify Istiod Pods.

Note: Inspect the istiod pod logs to verify that they are free from errors. Generally, all of the “info” level logs should be ignored. The “warn” level and “error” level logs are usually significant.

```

# kc -n fed-istio get pods

# kc -n fed-istio logs <Istiod_Pod>

```

Example of two “info” level logs are:

```

2022-06-30T14:13:56.933549Z    info    ads    SDS: PUSH

request for node:istio-ingressgateway-li-9b6c494b8-
xznj8.fed-smf resources:0 size:0B cached:0/2

2022-06-30T14:16:05.402650Z    info    ads    ADS:

"192.168.1.50:36326" goodbye-world-3-2-0-346-main-int-dbg-
8598748c47-421ch.fed-example-1155 terminated rpc error: code
= Canceled desc = context canceled

```

9. Verify ExternalHost resources.

ExternalHost resources define the access to hosts outside of the K8s cluster. Some applications (For example: li-x1, li-x2) dynamically create “ExternalHost” resources using the istio-cfg-agent APIs.

There may be error scenarios where connectivity to the External Host is not functional. In those scenarios, verify the successful creation of the external hosts (assuming the external host is operational and reachable).

Verify the ExternalHost resource was created, and that the “status” section has been populated.

Verify that the IstioCfgAgt logs in the NF namespace does not indicate errors creating the ExternalHost. The logs should be error free.

Verify that the Istio-Config_Controller in fed-paas-helpers recognized the ExternalHost resource and created the secondary resources. The logs should be error free.

```
# kc get externalhosts -A

# kc -n fed-XYZ logs <IstioCfgAgent_POD> -c istio-config-agent

# kc -n fed-paas-helpers logs <Istio-Config-Controller_POD>
-c istio-config-controller
```

Debugging Tools

Enable Istio-Proxy debug logs

If a specific pod is having issues with Service Mesh traffic, it can be useful to increase the istio-proxy container log verbosity. Increased verbosity will have a performance impact.

The default level is “info”. Typically, “debug” level is sufficient to observe most issues. Rarely it might be necessary to set to “trace” level. Note that trace level is extremely verbose and will have a larger performance impact.

```
# kc -n NAMESPACE exec -it POD -c istio-proxy -- curl -v -X
POST http://127.0.0.1:15000/logging?level=debug

# kc -n NAMESPACE exec -it POD -c istio-proxy -- curl -v -X
POST http://127.0.0.1:15000/logging?level=trace
```

Restore the default log level for the istio-proxy container.

```
# kc -n NAMESPACE exec -it POD -c istio-proxy -- curl -v -X
POST http://127.0.0.1:15000/logging?level=info
```

Dump Istio-Proxy Configuration and Statistics

If a specific pod is showing issues in forwarding traffic on the Service Mesh, it can be useful to dump the configuration and the statistics from its istio-proxy container.

After the data is collected, utilize SCP to transfer the output file from the K8s cluster to the destination host for further analysis.

```
# kc -n NAMESPACE exec -it istio-ingressgateway-75c6fc4fdd-
ztmdw -c istio-proxy -- curl -v
http://127.0.0.1:15000/config_dump > someFileA

# kc -n NAMESPACE exec -it istio-ingressgateway-75c6fc4fdd-
ztmdw -c istio-proxy -- curl -v
http://127.0.0.1:15000/clusters > someFileB

# kc -n NAMESPACE exec -it istio-ingressgateway-75c6fc4fdd-
ztmdw -c istio-proxy -- curl -v
http://127.0.0.1:15000/listeners > someFileC

# kc -n NAMESPACE exec -it istio-ingressgateway-75c6fc4fdd-
ztmdw -c istio-proxy -- curl -v http://127.0.0.1:15000/stats
> someFileD
```

Dump Istio and Envoy Versions

Inspect the running versions of Istio, as Istio is packaged with the UC release.

```
# kc -n NS exec -it POD -c istio-proxy -- ps -ef

UID      PID      PPID      C STIME TTY          TIME CMD istio-
p+         1         0 0 16:58 ?        00:00:09

/usr/local/bin/pilot-agent proxy sidecar --domain fed-
smf.svc.cluster.local --proxyLogLevel=warning --
proxyComponentLogLevel=misc:error --log_output_
level=default:info

istio-p+   16      1 0 16:58 ?        00:01:04

/usr/local/bin/envoy -c etc/istio/proxy/envoy-rev0.json --
restart-epoch 0 --drain-time-s 45 --drain-strategy immediate

--parent-shutdown-time-s 60 --local-address-ip-v

istio-p+   38      0 0 19:49 pts/0    00:00:00 ps -ef

# kc -n NS exec -it POD -c istio-proxy --

/usr/local/bin/pilot-agent version

version.BuildInfo{Version:"1.11.2",
GitRevision:"96710172e1e47cee227e7e8dd591a318fdfe0326",
GolangVersion:"go1.16.7", BuildStatus:"Clean",
GitTag:"1.11.2"}

# kc -n NS exec -it POD -c istio-proxy --

/usr/local/bin/envoy -version
```

```
/usr/local/bin/envoy version:
56534e716b8f7f5caad0f852b79680d97934717f/1.19.1/Clean/RELEASE/BoringSSL
```

Istio Logs

These commands illustrate how to manually view the container logs for the istio-operator and istiod pods.

```
# kubectl -n fed-istio get pods

# kubectl -n fed-istio logs <istio-operator-pod> -c istio-operator

# kubectl -n fed-istio logs <istiod-pod> -c discovery
```

Istio-Proxy Logs

These commands illustrate how to manually view the istio-proxy container logs from any running application pod.

```
# kubectl -n fed-XYZ get pods

# kubectl -n fed-XYZ logs POD_NAME -c istio-proxy
```

Kargo Collection

A Kargo log capture will be needed if fed-istio fails or if there are issues with auto-injected istio-proxy containers.

Execute the capture beginning 60m prior to the onset of the failure event. Capture logs from fed-istio and from the effected federation(s). The two examples provided below can be modified or extended to cover the affected federation(s).

To capture Kargo logs from fed-smf and fed-istio, execute below command from the masterVM.

```
# /usr/local/bin/kubectl-afn-kargo-collect -p fed-smf:kargo-default-profile,fed-istio:kargo-default-profile -d

<duration-in-minutes> -s <UTC starttime in format- 2022-04-14T15:16:00.00Z>
```

To capture Kargo logs from fed-upf and fed-istio, execute below command from the masterVM.

```
# /usr/local/bin/kubectl-afn-kargo-collect -p fed-upf:kargo-default-profile,fed-istio:kargo-default-profile -d

<duration-in-minutes> -s <UTC starttime in format- 2022-04-14T15:16:00.00Z>
```

Troubleshooting Kibana

This section provides troubleshooting topics for Kibana.

Error while Creating an Index Pattern in Kibana

This section describes the procedure to resolve an error while creating an index pattern in Kibana.

Prerequisites

None

Issue Description

The error is FORBIDDEN/12/index read-only / allow delete (api).

Possible Causes

None

Troubleshooting Steps

None

Workaround

Go to the dev tools console in Kibana GUI and run the below API:

```
PUT .kibana/_settings
{
  "index": {
    "blocks": { "read_only_allow_delete": "false"
  }
}
```

Verification Steps

None

Kibana Index Query and Creation Failure in Elastic

This section describes the procedure to resolve Kibana index query and creation failure in Elastic.

Prerequisites

None

Issue Description

This error occurs when the Elasticsearch goes down and comes up when Kibana is still up and running. In this case, Kibana does not automatically recover when the ES cluster goes from RED to GREEN but requires a restart of the Kibana pod.

Possible Causes

None

Troubleshooting Steps

None

Workaround

Upgrade the Kibana using the following command:

```
helm3 upgrade fed-kibana --namespace fed-kibana --create-namespace <chart-path> --set pod-kibana.upgrade.version="<unique-id>"
```

Note: The `pod-kibana.upgrade.version` must be unique for every upgrade.

Verification Steps

None

Unable to Create an Index Pattern from Kibana

This section describes the procedure to resolve an error to create an index pattern in Kibana.

Prerequisites

None

Issue Description

Kibana is stuck while creating an index pattern. Kibana stays read-only when ES high-disk watermark is exceeded even though later it has gone below the limit.

Possible Causes

None

Troubleshooting Steps

None

Workaround

Go to the dev tools console in Kibana GUI and run the below API:

```
PUT /_all/_settings
{"index.blocks.read_only_allow_delete": null}
```

Verification Steps

None

DebuggingPodRestart

This section provides steps to troubleshoot the restart of NF pods in Unity Cloud. It is mainly applicable for in-house built pods like smf-cc, amf-cc and others but not for the open-source components.

Possible Causes

The most widely observed reasons for abrupt pod restart are as follows:

1. Process crash within the container
2. Container hitting OOM error
3. Liveness/startup probe failures
4. Graceful process exit within the container

Category: **System**

Irrespective of the reason which results in a pod restart, it is the infra container which triggers the pod restart by invoking the pod delete REST API to the Kubernetes API server.

- Verify the restarted pod application logs to check if it indicates any reason for the restart.
- Check the output of the command to see if there are any events indicating the pod restart.
- Disable the pod restart and monitor the application/infra container logs.

Symptoms

The following are the symptoms observed on pod restart:

1. Verify if there are ELF core files getting generated related to the restarted application.
2. Investigate the infra container logs of the restarted pod for any of below logs:
 - "Container Termination Detected! filename: \$filename". This log indicates if the pod got deleted either due to reason (1) or (3)
 - "Triggering pod termination due to restarted container. Container Name: \$containerName". This log indicates that the pod got restarted due to one of the containers hit the OOM error.
3. Verify the below event indicating the container's liveness probe failing resulting in pod restart:
 - 6m13s Normal Killing pod/ippe-2-3-0-828-main-int-547c9fc764-srbdq Container kafka-client failed liveness probe, will be restarted
4. NF Virtual Restart is triggered.

Artifacts

The following are the artifacts to be considered:

- Logs of all the pods especially the logs of the restarted pod.

- Output of “kubectrl get events -n <namespace>” for the timestamp when the pod was restarted.
- Any ELF core files or breakpad minidump files which got generated.

Troubleshooting Steps

Data Collection

Trigger the Kargo collection using the required profile for the collection of required data.

Data Analysis

- Based on the data collected, verify if the pod restart happened because of any of the reasons mentioned above.
- If there are any ELF core files generated, use the “cna coredump” command to generate the backtrace for the further analysis of the application failure.

Troubleshooting Common Microservices

This section includes troubleshooting common microservices topics.

CHF Agent

- [Usage Report Not Sent to or Received by CHF Server](#)

Configuration Manager

- [Application Pods are in n-1/n State](#)
- [Certificate Issues for RESTCONF Requests](#)
- [Cfgmgr Failure](#)
- [CLI Login Issues](#)
- [Config Update/Notification Not Received at an Application](#)
- [Incorrect Display of CLI Command or Help](#)

GTPC Agent

- [Peer Initiated Traffic Dropped by Gtpcagent](#)

CHF Agent

UsageReportNotSenttoor ReceivedbyCHFServer

Issue Description

Usage report not sent to or received by CHF server.

Possible Causes

Following are the possible causes:

- Misconfiguration of FQDN in CHF profile.
- Kafka replay profile not associated with CHF server profile.
- Kafka cluster is down:
 - If FQDN is correctly configured in chf-server-profile and the kafka replay profile is associated with chf-server-profile:
 - if CHF server is down (get failure response, one time report will be generated).
 - if kafka cluster is down then usage report will not be sent and unaccounted report will be generated.

Troubleshooting Steps

Following are the troubleshooting steps:

MisconfigurationofFQDNinCHFprofile:

Run the below command from master node terminal prompt to login inside the cfgmgr container and get config mode access to check running-config:

```
kubectl exec $(kubectl get pods -n fed-smf | grep cfgmgr |  
awk '{print$1}') -n fed-smf -c cfgmgr -it bash --confd_cli  
-C -u admin -g admin
```

Figure 1 Command Output

```
admin@pod-cfgmgr-3-2-0-21-rel-3-1-1-dbg-5db95f49b6-krhs8 04:26:11# show running-config chf-agent chf-server-profile
chf-agent chf-server-profile CHF-SERVER-PROFILE1
admin-state UP
fqdn http://10.163.225.212:80
port 80
schema HTTP
!
chf-agent chf-server-profile SERVER1
admin-state UP
fqdn http://10.163.225.212:80
port 80
schema HTTP
!
chf-agent chf-server-profile SERVER2
admin-state UP
fqdn http://10.163.225.197:8080
port 80
schema HTTP
!
chf-agent chf-server-profile SERVER3
admin-state UP
fqdn primary.com
port 80
schema HTTP
!
chf-agent chf-server-profile SERVER4
admin-state UP
fqdn www.replay.com
port 80
schema HTTP
!
chf-agent chf-server-profile SERVER5
admin-state UP
fqdn secondary.com
port 80
schema HTTP
!
admin@pod-cfgmgr-3-2-0-21-rel-3-1-1-dbg-5db95f49b6-krhs8 04:26:22#
```

Check if the FQDN is configured and reachable.

KafkareplayprofilenotassociatedwithCHFserverprofile

Verify from the logs if CHF server is notified as down or if there are blacklisting:

- "NchfAgentEvent":"Server added to BlackList","Server":http://<ip>:80
- "eventName":"Nchf Unaccounted Usage Report Event","message":"Unaccounted Usage Report:< Usage report >"

If none of the counters or logs shows that it has been sent to kafka. Check the associated Prometheus counters.

Check if the kafka replay feature is enabled and the kafka replay profile configuration is associated with the CHF server profile.

Run the below command from master node terminal prompt to login to cfgmgr container and get config mode access to check running-config:

```
kubectl exec $(kubectl get pods -n fed-smf | grep cfgmgr |
awk '{print$1}' ) -n fed-smf -c cfgmgr -it bash --confd_cli

-C -u admin -g admin
```

Figure 2 Command Output

```
admin@pod-cfgmgr-3-2-8-21-rel-3-1-1-dbg-5db95f49b6-krhs8 04:11:46# show running-config chf-agent chf-profile replay-profile REPLAY-PROFILE-1
chf-agent chf-profile CHF-PROFILE-2
  primary-chf-server-profile SERVER1
  secondary-chf-server-profile SERVER2
  http-request-timeout 5
  replay-profile REPLAY-PROFILE-1
!
admin@pod-cfgmgr-3-2-8-21-rel-3-1-1-dbg-5db95f49b6-krhs8 04:12:08# show running-config chf-agent chf-replay-profile REPLAY-PROFILE-1
chf-agent chf-replay-profile REPLAY-PROFILE-1
  partition-count 5
  replication-factor 1
  retention-period 684800000
  retention-size -1
  replay-window-size 2
!
admin@pod-cfgmgr-3-2-8-21-rel-3-1-1-dbg-5db95f49b6-krhs8 04:13:35#
```

Kafkaclusteris down

Verify from the logs if kafka is notified as down and see if there are kafka down related failure logs like below:

- "ReplayMgrEvent": "Received Kafka Topic Response", "TopicName": "< Topic name>", "HttpStatusCode": 503
- "URI": "/convergedcharging/v2/chargingdata/onetimereport"
- "eventName": "Nchf Unaccounted Usage Report Event", "message": "Unaccounted Usage Report:< Usage report >"
- "KafkaAgentEvent": "Received response from Kafka Client", "HttpStatusCode": 503
- "KafkaAgentEvent": "Failure in sending message to Kafka Client"

Check if there is any hardware or software issue due to which kafka cluster is down.

Workaround

None

Verification Steps

- Misconfiguration of FQDN in CHF profile: FQDN should be configured under chf-server-profile.
- Kafka replay profile not associated with CHF server profile: Kafka replay profile should be configured under replay-profile.

Configuration Manager

This section provides troubleshooting topics for Configuration Manager.

Application Pods are inn-1/nState

This section describes the troubleshooting steps to resolve application pods showing in n-1/n state and not moving to n/n state.

Prerequisites

Collect the logs from the following folders:

- cfgmgr logs
- /var/log/messages from cfgmgr pod
- Infra container logs from the application
- fileserver pod logs

Issue Description

Application pods are waiting for the cfgmgr update to move to n/n state.

Possible Causes

The following are the possible causes for the CLI login issues:

- Application pod is unable to download the json file in cfgSc sidecar due to consul or file server issues or due to connection issues.
- Config update notification not reached/missed at the cfgSc sidecar.

Troubleshooting Steps

The following are the troubleshooting steps for application pods n-1/n state issue:

- Verify the cfgmgr logs to check why Schema Manager has not received any schema upload notifications.
- Verify any http errors while performing http get on the schemas uploaded.
- Verify if there were any upgrade failures.
- Verify if there were any confd failures during the processing of the config by checking:/var/log/messages in cfgmgr pod.
- Verify for any download failures of the json file in cfgSc sidecar which are uploaded from cfgmgr and also cross check any communication failures from the infra container logs in application.

Workaround

- Delete pod-cfgmgr
- Delete the application pod to see if it results in application pod status to moving to n/n state.

Verification Steps

Verify that the status of the pods are moved to n/n state with above workarounds.

Certificate Issues for RESTCONF Requests

This section describes the troubleshooting steps to resolve Certificate issues during the RESTCONF curl requests.

Prerequisites

Collect the logs from the following folders:

- cfgmgr logs
- /var/log/messages from cfgmgr pod
- secrets created in application namespace after application comes up using the command “`kc get secrets -n <application-namespace>`”

Issue Description

Certificates are not working for RESTCONF curl requests.

Possible Causes

The following are the possible causes for the TLS certificates issue:

- Certificates are not populated either by using User-defined certs values or cert manager.
- Local users are not present that are mandatory for RESTCONF curl request.
- Certificates validity expired.
- Certification authority and Client certificates are not matching.
- Input certificates supplied for the RESTCONF curl are not valid.

Troubleshooting Steps

The following are the troubleshooting steps for Certificate issues for RESTCONF request:

- Verify the command syntax for the RESTCONF with all the parameters with valid values like ca-crt, tls-crt and tls-key (check the validity and if there are any other issues with the input values).
- Verify that the secrets are created using the command `kcgetsecrets` for the namespace.

Workaround

Populate the new client certificates from the CA that is present for the cfgmgr and verify the issue is resolved.

Verification Steps

Verify that the curl request works for RESTCONF with above workaround.

Cfgmgr Failure

This section describes the troubleshooting steps to resolve cfgmgr failure.

Prerequisites

Collect the logs from the following folders:

- crash dump file from the folder /filestore/cores/<fed-name> on OPM
- cfgmgr logs
- /var/log/messages of cfgmgr pod

IssueDescription

The cfgmgr pod fails during upgrade/configure/running system.

PossibleCauses

The following are the possible causes for configuration manager failure:

- cfgmgr gets stuck in processing and due to unresponsiveness, heartbeat monitor process force shuts the cfgmgr after default timeout of 60 seconds.
- cfgmgr has memory corruption/violation.

TroubleshootingSteps

The following are the troubleshooting steps for the cfgmgr failure issue:

- Collect the crash dump from the filestore and decode to know the failure reason.
- Disable the cfgmgr pod restart by editing the cfgmgr pod deployment file using the flag CNA_DISABLE_PROCESS_MONITORING.

Workaround

None

VerificationSteps

- Check if the process is continuously failing (even after cfgmgr reboot after failure).
- Check if process comes up after failure.

CLI LoginIssues

This section describes the troubleshooting steps to resolve CLI login issues.

Prerequisites

Collect the logs for the following folders:

- cfgmgr logs

- /var/log/messages of cfgmgr pod
- /etc/pam.d/tacacs and /etc/pam.d/sshd file contents of the pod-cfgmgr
- helm values dump (for the configurations)

IssueDescription

User unable to login into CLI using the username/password (local users or tacacs external users).

PossibleCauses

The following are the possible causes for the CLI login issues:

- Local users are not loaded during the bootup
- Incorrect configuration is supplied from the values.yaml
- Incorrect privileges set for the user (for both local users and external tacacs users)

TroubleshootingSteps

The following are the troubleshooting steps for CLI login issue:

- Verify the /etc/passwd file, to see if users present both in vm and cfgmgr pod.

Example: for local users.

```
cat/etc/passwd|grepoper
```

```
operator:x:11:0:operator:/root:/sbin/nologin
```

```
oper:x:1100:1100::/home/oper:/usr/cna/cna-  
cfgmgr/cfgmgr/bin/cli
```

```
lioper:x:2100:2100::/home/lioper:/usr/cna/cna-  
cfgmgr/cfgmgr/bin/cli
```

- Verify that the /etc/securitypolicy.json file is created by checksecurity script for both local and external (tacacs) users, located at /etc/config/security-config-map/path in pod-cfgmgr.
- Verify the tacacs configurations (external) in /etc/tacplus.conf by logging into cfgmgr pod.
- Verify if there are any error logs printed during the bootup in cfgmgr pod logs.
- Verify the /var/log/messages by exec into pod-cfgmgr for any logs related to tacacs server like connection failures.

Workaround

- Restart/Delete pod-cfgmgr pod.
- Edit the /etc/pam.d/tacacs file and /etc/pam.d/sshd file with the sample configurations and restart sshd process.

VerificationSteps

Verify login with the above steps/workarounds.

Config Update/Notification Not Received at an Application

This section describes the troubleshooting steps to resolve config update not received at the application.

Prerequisites

Collect the logs from the following folders:

- cfgmgr logs
- /var/log/messages from cfgmgr pod
- Infra container logs messages for the application.
- fileserver pod logs

IssueDescription

Config/notification update not received at the application after configuring through the CLI or during the bootup.

PossibleCauses

The following are the possible causes for the config update not being received at application:

- cfgmgr unable to upload the json file to fileserver (due to connection/fileserver issues)
- cfgSc unable to download the json file (due to connection or fileserver issues)
- http issues
- consul issues

Troubleshooting Steps

The following are the troubleshooting steps for the config update issue at application:

- Verify the cfgmgr logs to check if there were any issues during the uploading of configuration file.
- Verify the cfgSC logs to check if there were any issues during the downloading of configuration file.
- Verify that the config commit notifications reached the cfgSc.
- Verify that the config update notification was sent from cfgSC to application.

Workaround

- Delete pod-cfgmgr.

- Delete the application pod to see if it resolves the issue in cfgSC container.

VerificationSteps

Verify that the configuration update using show/show-running cli command and see if it is resolved with the above workaround.

IncorrectDisplay of CLI Commandor Help

This section describes the troubleshooting steps to resolve incorrect display of CLI command or help.

Prerequisites

Collect the logs from the following folders:

- cfgmgr logs
- /var/log/messages of cfgmgr pod

IssueDescription

After upgrade, incorrect/hidden command or help is displayed.

Possible Causes

The following are the possible causes for the display of incorrect/hidden command or help:

- Updated schema not loaded due to cfgmgr errors.
- Command is explicitly hidden in the -initial.xml file (restricted privileges).
- Upgrade failed, resulting in not loading of updated schema (schema upgrade failed scenario).
- Schema from the older files are loaded (older files not removed completely), this results in not uploading the schema that are expected and hence appearing as per the old schema files.

Troubleshooting Steps

The following are the troubleshooting steps for the display of incorrect/hidden command or help:

- Verify the exact version of the schema, expected using the CLI command, show cfgmgr schema and verify it is loaded with expected version.
- Verify the cfgmgr logs to identify if any upgrade failed.
- Verify the -initial.xml file is loaded as expected or failed while loading.

Workaround

1. Upload the schema file again, if the loading failed during the previous boot-up. *Example: # script reload*
2. Delete the application pod and verify if it loads the schema.
3. Delete the cfgmgr pod and verify if it loads the schema.

Verification Steps

Verify the command/help to see the issue is resolved.

GTCP Agent

Peer Initiated Traffic Dropped by Gtpcagent

Peer initiated traffic dropped by gtpcagent with the log statement Nogtpcinterfacefound.

Issue Description

Peer initiated traffic dropped by gtpcagent with the log statement Nogtpcinterfacefound.

Possible Causes

Configuration on the CLI is missing for the relevant gtp-c interface where the message is received. In case of an external service supporting multiple interface types, each such interface type has to be configured on CLI. Otherwise, The interface will not be identified.

Troubleshooting Steps

1. Execute showrunning-configgtpcagent to view the interface configuration.

```
gtpcagent gtpc-interface s11_mme
interface-type S11-MME
n3-request 3
t3-response 5000
version v2
echo-interval 0
t3-echo-response 15000
n3-echo-request 3
inactivity-timer 5
t3-path-recovery-echo-response 50000
dscp-marking af12
consumer-retry-count 3
!
```

Note: In this example, s5_s8_pgw interface is missing.

2. To configure s5_s8_pgw interface, execute **gtpcagent gtpc-interface s5_s8_pgw**.

```

interface-type      S5-S8-PGW
n3-request          3
t3-response         5000
version             v2
echo-interval       0
t3-echo-response    15000
n3-echo-request     3
inactivity-timer    5
t3-path-recovery-echo-response 50000 dscp-
marking             af12
consumer-retry-count 3
associated-retry-config  retryProf

```

3. To save, type **commit**.

Verification Steps

- Show CLI command output after adding the new interface configuration. Below is an example [here s-5-s8-pgw interface configuration is now present]:

```

show running-config gtpcagent

gtpcagent gtpc-interface s11_mme
interface-type      S11-MME
n3-request          3
t3-response         5000
version             v2
echo-interval       0
t3-echo-response    15000
n3-echo-request     3
inactivity-timer    5
t3-path-recovery-echo-response 50000
dscp-marking        af12
consumer-retry-count 3

```

```

!
gtpcagent gtpc-interface          s5_s8_pgw
interface-type  S5-S8-PGW
n3-request      3
t3-response     5000
version v2
echo-interval   0
t3-echo-response      15000
n3-echo-request 3
inactivity-timer      5
t3-path-recovery-echo-response 50000 dscp-
marking      af12
consumer-retry-count   3
associated-retry-config  retryProf
!

```

- To confirm whether the traffic is supported after the configuration, execute **kc get svc -n <fed-name>**. If required service is not present, then the necessary service supporting the required interface type should be configured and then deployment upgraded.

Note: This should be followed by restart of all gtpcagent pods.

- Peer initiated traffic is now supported by gtpcagent and the log No gtpc interface found will no longer be visible.
- Verify that the peer initiated traffic is being handled correctly with the help of stats: gtpc_agent_request_received and gtpc_agent_response_sent.
- Below are some of the snapshots of the statistics to be checked as verification for peer initiated traffic.

Figure 4 gtpc_agent_request_received

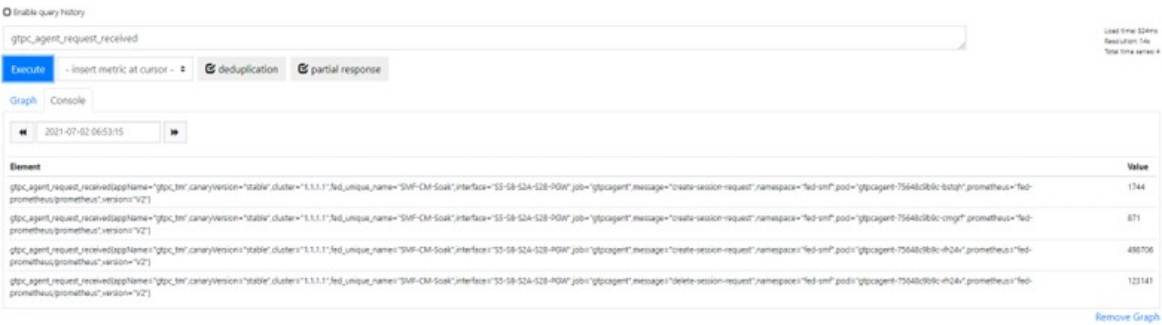


Figure 5 gtpc_agent_response_sent

